

Note: There are 3 projects of different lengths; the first one is far shorter.

The labs use redpitaya board with a daughter board with switches, a button and a 7-segment display. The appendices give some information about simulation using ghdl and gtkwave; synthesis using in particular yosys and programming using openFPGALoader. The end of each projects will use Vivado to configure a processor and have a link between the Processing System (PS) on linux and the Programmable Logic (PL).

You only need to install on your computer ghdl and gtkwave to work at home (since you do not have the board).

In the lab PCs, use as a working directory: home/etudiant/ELISE2/your_name.

Put useful comments in your VHDL files (no description but the purpose). In the report, you have to **explain** your choices, all the things you learned and why the simulations show it works correctly. Attach to the report all the .vhd files (including the testbenches) **only them**. Do not forget to simulate all components before assembling them.

Reports are due to before the 3rd lab for the 1st project, before the 5th lab for the 2nd one, and 2 weeks after the last lab for the 3rd one.

The use of ChatGPT or any other AI is strictly prohibited.

PROJECT 1: DDS

In order to better understand the working operation of the AD 9834 DDS that you will communicate with in the 3rd project, you will synthesize a DDS on a FPGA. Simply put, a DDS is composed of 3 parts: the **phase accumulator** (therefore an adder), an offset generator (another adder) and some memory containing the shape of the desired signal.

The DDS output frequency is:

$$f_{out} = f_{MCLK} * W/2^N$$

where f_{MCLK} is the master clock of the DDS (here the FPGA 125 MHz) and N the size in bits of the word W which adjust the output frequency. A DDS uses the fact that the phase is intrinsically linear: at each time increment Δt , the phase increase with the same value (modulo 2π) linked to the frequency: $\Delta\phi = 2\pi f \Delta t$. If the phase is coded in N bits (2^N corresponding to 2π), then W corresponds to the phase increase. The phase accumulator is then designed as an adder, which increases the coded phase with a W value. The memory (here RAM) allows the link between a phase and the corresponding amplitude of the signal according to its shape.

For this project, N will be a parameter (GENERIC before PORT) with a default value of N = 10. The signal shape will be a sinus, quantified with 14 bits (unsigned: 0 to $2^{14}-1$). The inputs will therefore be W, the phase offset (10 bits) and the clock. The output is a 14-bits signal.

1) Use octave software to compute the signal shape stocked in the memory and that you will copy/paste into your .vhd file. Write your vhdl description of the DDS.

Write the test bench and simulate. In your simulations with gtkwave, you can choose the format “analog” to see the “shape” of your output.

Study the result with different value of W. What happens if $W > 2^9$? Explain with a graph in timing and another one in frequency. What is the largest value of the output frequency f_{out} ? What is the purpose of the offset, which adds some value to the phase obtained from the phase accumulator? In particular, what is its obvious purpose when there are several DDS?

2) Before programming the board, you need to write an encapsulating .vhd file. Check the ADC_DAC_channel1.vhd example to know how to use the 125 MHz DAC on the red pitaya board. You will set the offset to 0 and set 2 bits of W directly in the encapsulating file,

the others will be linked to the switches (do not forget to put the jumpers of the board for the GPIO pins).

Before using the makefile `makefile.openXC7`, **source /home/jmfriedt/openxc7/export.sh**

Test by visualizing on an oscilloscope several values of W. You can modify if you want the clock in the encapsulating file to slow down the signal.

3) For the last part of this lab, you will aim to log into the linux on the red pitaya and directly change W in order to have all the possibilities for W and the offset, and not be limited by the number of switches. To configure the interface between PS and PL you will use Vivado software from Xilinx.

To launch Vivado:

type « `export LANG=en_US.UTF-8` »

then « `source /home/jmfriedt/Xilinx/Vivado/2024.2/settings64.sh` »

finally « `vivado &` »

Open *TP_M2_source* project in `home/etudiant/seb` and save it (from vivado) into your folder. This project includes the processing system, a port (AXI_GPIO) to link PS and PL and the converters management.

Note: if a window is hidden, you can obtain it with the menu « Window » at the top. Except for *Project Manager*, to change « settings » of the various grouping in the « Flow Navigator » (to the left), you have to right click on the grouping.

Open the block design (Flow Navigator → IP Integrator). You can undock the *Diagram* window with the *Float* icon  at the top right. Right-clicking in this window enables you to create ports (*Create Port*), to add modules that you have already designed (*Add Module*) and to add modules from proprietary libraries (*Add IP*). Study the global operation of this project.

Add your *DDS.vhd* file (without the encapsulation) in your project with File → Add sources or by clicking on the « Add Sources » icon in the « Project Manager » part of the « Flow Navigator » (top left). In the diagram window, add your *DDS.vhd* file (add Module). If you modify the source file, you need to refresh the module (right click on it and select refresh module). You can change the names of the instances by selecting them, then with the *Properties* window (under the *Sources* window).

Double click on the GPIO component and select “enable Dual Channel” to have 2 output ports. Change the GPIOs width to 10. Click OK to confirm the choice and close the window. Click on the AXI bus « + » of the GPIO component to see all the inputs/outputs. You can connect the outputs `gpio_io_o` to your DDS inputs by click and drag. The `s_axis_tdata` of the DAC component needs 32 bits. You can add the *constant* IP and the *concat* IP to fill the MSb with zeros. Connect a constant ‘1’ to the input `s_axis_tvalid`. Modify the *axis_red_pitaya_dac.vhdl*:

Replace “`int_dat_a_reg <= int_dat_a_wire(DAC_DATA_WIDTH-1) & (not int_dat_a_wire(DAC_DATA_WIDTH-2 downto 0));`”

with “`int_dat_a_reg <= int_dat_a_wire(DAC_DATA_WIDTH-1 downto 0);`”

Explain the reason of this change knowing the output of the DAC is $\pm 1V$. Do not forget to refresh the module. Save the project.

Click on *validate design* icon (check icon ) to check problems (the warning about the clock for the ADC is ok).

Generate the bitstream (from the *Flow Navigator*), all the previous necessary synthesis steps will be automatically performed. This bitstream creation can take a lot of time, so you should continue.

Another way to program the FPGA is directly from the linux on the chip, without using a JTAG probe. In your project folder on the computer, write with a text editor the .bif file for the creation of the .bit.bin file:

```
all:
{
    project_path/ project_name.runs/impl_1/bistream_name.bit
}
```

You will be able to use bootgen to create the .bit.bin file when the .bit file is finally made:
/home/jmfriedt/Xilinx/Vivado/2024.2/bin/bootgen -image bif_file.bif -arch zynq -process_bitstream bin

With a terminal, connect to the red pitaya board via ssh (after plugging everything...):

```
ssh root@192.168.2.200
```

```
(pwd root)
```

Within another terminal, copy the .bit.bin file into the red pitaya folder /tmp (if the .bit.bin is not yet generated, use in the meantime the one in /home/etudiant/seb/docs):

```
scp fichier_a_copier root@192.168.2.200:/tmp
```

With the red pitaya prompt, copy the .bit.bin file into /lib/firmware

Note that you can also communicate with the red pitaya board using minicom with the following parameters /dev/ttyUSB0, 115200 8N1, no flux correction (minicom -s to change the configuration) and use the USB as a serial communication instead of ethernet.

From the red pitaya board, flash the FPGA:

```
echo nom_du_bitstream.bit.bin > /sys/class/fpga_manager/fpga0/firmware
```

For your information, fpga_manager homogeneize the management of FPGA within linux and replace the former Xilinx FPGA configuration using xdevcfg.

With vivado, when the *Diagram* window is displayed, you can read the GPIO memory space in the Address Editor (should be 0x4120_000).

AXI_GPIO datasheet tells the following offsets:

0: GPIO_DATA

4: GPIO_TRI (1 → input, 0 → output)

8: GPIO_DATA2

0xC: GPIO_TRI2

0x11c: GIER

0x128: IP IER

0x120: IP ISR

From the red pitaya, you can read the data at address 0x41200000 (for instance) with:

```
devmem 0x41200000 w
```

(w = word = 32 bits, you can also use h = 16 bits and b = 8 bits)

To write XXXX into the address 0x41200000: devmem 0x41200000 h XXXX

Modify GPIO_TRI and GPIO_TRI2 to assign the 2 GPIO as outputs.

Modify GPIO_DATA and GPIO_DATA2 to change W and the offset. Check the effect with an oscilloscope. You need to use a 50 Ω load to avoid bouncing.

APPENDICES 1: Simulation with open tools

As a habit, use the same name for the entity and the .vhd file.

To analyze and see potential errors, type in a terminal:

```
ghdl -a nameofthefile1.vhd nameofthefile2.vhd
```

In the case of the use of a “work” package, put the package file first.

In order to use standard 2008 VHDL (which allows /* */ for several lines commentaries for instance):

```
ghdl -a --std=08 nameofthefile.vhd
```

If there is no error, the prompt will display nothing but the file work-obj93.cf (or work-obj08.cf for VHDL 2008) will be created.

After writing the test_bench (see example in appendices 3), you can check to combination of the two in the same way (and see potential errors in the test_bench):

```
ghdl -a testbench.vhd nameofthefile.vhd
```

To create the simulation file: `ghdl -r entity_test_bench options --wave=name_simu_file.gwh` for instance: `ghdl -r simu --stop-time=100ns --wave=wave.gwh`

Note that you need to use `ghdl -r --std=08` if you used `ghdl -a --std=08`

Launch gtkwave with: `gtkwave name_simu_file.gwh`

Click on the entities or signals in the top left window to make signals appear in the *signals window* (just bellow the previous window). Then click in this *signal window* on the signals you want to visualize in the *wave window*.

APPENDICES 2: Programming with open tools

Names of the inputs and outputs declared in the entity should correspond to the names in the ports.xdc file (case sensitive). Even if you use only one bit of a vector (one switch for instance), use a 2-bit std_logic_vector. In case of using the 125 MHz redpitaya clock you need to use the components IBUFDS and BUFG (refer to appendices 4 of M1 lab subject or filADC_DAC_channel1.vhd, no need to use special files for these components). To comply with these two requirements without modifying the .vhd file you simulated you can create another .vhd file to encapsulate it (using component/Port Map).

Type once **`source /home/jmfriedt/openxc7/export.sh`**

Modify the makefile that you can find in moodle (makefile.openXC7).

PRJ corresponds to the name of the project but also the name of the .vhd file with the top level entity, which also must be the name of this top level entity.

VHDL_BIBLI corresponds to all the other .vhd files needed (can be left blank if there is no other .vhd file)

V_LIST should contain project_name.v (verilog file which will be created from the .vhd files). It could also contain other verilog files but that won't be our case.

CONSTR contains the constrain files, in our case ports.xdc

When typing `make -f makefile.openXC7`, you may have on some PC the error “ImportError: cannot import name 'antlr_to_tuple” but it still should work (the size of the .bit file should be several Mo).

yosys with the ghdl plugging will convert the VHDL .vhd files into a verilog .v file.

yosys will then make the RTL (Register Transfer Level) synthesis step.

nextpnr will make the place and route steps. Only some FPGA are supported with this non-proprietary tool.

Thanks to Project X-ray (fasm2frames, xc7frames2bit) the bit-stream with the right format for Xilinx XC7 FPGAs can be created (.bit).

`make load` will use openFPGAloader to program the FPGA thanks to the JTAG H2S programming cable from Digilent. The JTAG connector is situated near the Ethernet plug of the redpitaya with the names of the pins (3V3, GND, TCK, TD0, TDI, TMS) written under the board.

Do not forget to perform `make clean` before performing `make`.

APPENDICES 3: Example of testbench

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity simu is
-- Port ( );
end simu;

architecture Behavioral of simu is
    component entity_to_simulate is
        Port ( clk : in STD_LOGIC;
              SW : in STD_LOGIC_VECTOR(3 downto 0);
              Button : in STD_LOGIC;
              led : out STD_LOGIC_VECTOR(7 downto 0) );
    end component;
    signal clk: STD_LOGIC;
    signal SWt : STD_LOGIC_VECTOR(3 downto 0);
    signal buttont : STD_LOGIC;
    signal ledt : STD_LOGIC_VECTOR (7 downto 0);
    constant PERIOD : time := 8 ns; -- 125 MHz

    begin
        tb: entity_to_simulate PORT MAP(clk=>clk, SW => SWt, Button=> buttont,
        led => ledt);

        clk_process: process
        begin
            clk <= '0';
            wait for PERIOD/2;
            clk <= '1';
            wait for PERIOD/2;
        end process;

        button_proc: process
        begin
            buttont <= '0';
            wait for 8 ns;
            buttont <= '1';
            wait for 10*PERIOD;
        end process;

        SW_proc: process
        begin
            FOR i IN 0 TO 15 LOOP
                SWt <= std_logic_vector(to_unsigned(i,4));
                wait for PERIOD;
            END LOOP;
        end process;
    end Behavioral;

```